



# AYUDA MEMORIA

Todo lo macro de la app ToDo en 3 capas — para repasar antes de la clase con Leo

Frontend

Backend

Base de datos

API REST

CRUD

JSON

## 1 · Las 3 capas de la app

### FRONTEND

lo que el usuario

**VE**

HTML + CSS + JS vanilla  
(1 pantalla, sin login)

### BACKEND

la

**LÓGICA**

recibe, calcula, guarda  
Express + API REST

### BASE DE DATOS

la

**MEMORIA**

SQLite local  
archivo tareas.db

▼ El frontend NUNCA toca la base de datos ▼

**Regla de oro:** Front = lo que ves · Back = lo que piensa · BD = lo que recuerda.

## 2 · Cómo se hablan (API + JSON)

- **API** = puente que permite que 2 sistemas se comuniquen aunque no hablen el mismo idioma.
- **API REST** = el tipo de API más usado en la web.
- **JSON** = el idioma de los datos: objetos `{ }` y arreglos `[ ]`.
- **Request** = lo que el front **manda** (petición).
- **Response** = lo que el back **responde**.

Front → manda JSON (request) → Back → consulta BD → Back → responde JSON (response) → Front pinta

**Desacople:** el backend no sabe si le habla una web, un celular u otro sistema. Solo recibe y responde JSON.

### 3 · CRUD — las 4 operaciones básicas

| Letra | Operación          | En tu app                |
|-------|--------------------|--------------------------|
| C     | Create — crear     | Agregar una tarea        |
| R     | Read — consultar   | Ver la lista de tareas   |
| U     | Update — modificar | Editar / cambiar estatus |
| D     | Delete — eliminar  | Borrar una tarea         |

Todo modelo de datos en una app se maneja con CRUD. Es la base universal.

### 4 · Los endpoints de TU app

| Método | Ruta            | CRUD       | Respuesta                  |
|--------|-----------------|------------|----------------------------|
| POST   | /api/tareas     | Crear      | 201 + tarea nueva          |
| GET    | /api/tareas     | Leer todas | 200 + lista                |
| GET    | /api/tareas/:id | Leer una   | 200 + tarea · 404          |
| PUT    | /api/tareas/:id | Modificar  | 200 + tarea · 400 inválido |
| DELETE | /api/tareas/:id | Eliminar   | 204 (sin cuerpo)           |

El backend se prueba con **curl** o Postman, sin necesidad del frontend. Eso se llama "cliente de API".

### 5 · El modelo "Tarea" (los datos)

#### Los 4 campos

- **nombre** — qué hay que hacer
  - **responsable** — quién la toma
  - **estatus** — en qué estado está
  - **descripcion** — detalle / notas
- (+ **id** = número único que da la base de datos sola)

#### Estatus = valores cerrados

-  **pendiente** — todavía no
-  **en progreso** — se está haciendo
-  **hecha** — terminada

El estatus NO es texto libre: solo estos 3 valores. Eso es normalización.

## 6 · Cómo levantar la app (2 pasos)

| Paso         | Qué hacer                                                       |
|--------------|-----------------------------------------------------------------|
| 1 · Backend  | Terminal en <b>backend/</b> → <b>npm start</b> → dejala abierta |
| 2 · Frontend | Doble clic en <b>frontend/index.html</b>                        |

⚠ **Orden importa:** primero el backend, después el front. Si no anda, la terminal dejó de decir "Backend corriendo".

package.json = el corazón del proyecto: ahí dice qué usa (dependencias) y cómo arranca (scripts → npm start)

## 7 · Los 4 que me confunden (¡ya no!)

| Yo decía...                | La verdad                                                   |
|----------------------------|-------------------------------------------------------------|
| "next" para el backend     | <b>Express</b> — las 3 famosas son Express, Fastify, NestJS |
| "mysql"                    | <b>SQLite local</b> — un archivo. MySQL fue la tarea vieja  |
| "create... y delete"       | <b>crear, consultar, modificar, eliminar</b> (CRUD)         |
| "no me acuerdo los campos" | <b>nombre, responsable, estatus, descripción</b>            |

## 8 · Reglas de oro del código

### const vs let

- **const** = no cambia (arreglos, objetos)
- **let** = cambia (contadores, acumuladores)
- **Pista de Leo:** "¿va a cambiar? → let"

### Selector de elementos

- **#** = id · **.** = clase
- `getElementById("id")` sin #

### Arreglo vs objeto

- **Arreglo** [ ] = lista, se agarra por posición [0]
- **Objeto** { } = ficha con nombre: valor, se agarra con punto .campo

### Otras

- **===** compara valor Y tipo (no ==)
- Punto y coma ; al final
- camelCase: **cargarTareas** (sin espacios, sin acentos)
- IDs con guion: **input-nombre**

## 9 · Vocabulario de Leo (para sonar técnico)

| Palabra                | Qué significa                                                    |
|------------------------|------------------------------------------------------------------|
| Middleware             | Función que va "en el medio" del pedido (ej. cors, express.json) |
| package.json           | Corazón del proyecto: dependencias + scripts                     |
| Desacople              | El back no sabe quién le habla                                   |
| Valores cerrados       | Campos con lista fija (el estatus solo puede ser 1 de 3)         |
| Fuente única de verdad | Los datos viven en UN solo lugar (la BD), no duplicados          |
| Ruta / endpoint        | La "dirección" de cada operación (/api/tareas)                   |

## 10 · Si Leo te pregunta algo que no sabés

"Esto todavía no me quedó firme, quiero entenderlo." — es lo que él quiere escuchar. Mejor que fingir.

- Validá que funcione ANTES de decir "listo" (probar de verdad).
- Entendé cada archivo: **server.js** (entrada) → **routes/tareas.js** (CRUD) → **db.js** (BD) → **js/app.js** (front que llama).
- El prompt al modelo: rol + contexto + tarea + formato + restricciones. Acotado, no "dame todo el código".